Building a Chinese Wall in Security-Enhanced Linux

by

Calum Cardownie

In partial fulfilment of the
requirements for the degree of
Bachelor of Science
in
CS408: Individual Project

University of Strathclyde Glasgow

Department of
Computer and Information Sciences

March, 2026

Abstract

The abstract is a bird's eye view of the project.

It should not exceed one page. Mention the scope and objectives of the project, the methodology used,the main findings, and significance of your results.

Acknowledgements

I would like to thank

Contents

1 Introduction **1**
1.1 Aims . . . 1
1.2 Objectives . . . 2
1.3 Report Format . . . 2

2 Background Knowledge **3**
2.1 The Chinese Wall (Brewer-Nash) Model . . . 3
2.2 Security-Enhanced Linux . . . 3
2.3 Bell-LaPadula model in SELinux . . . 3
2.4 Chinese Wall in UNIX . . . 3
2.5 Hypervisor-Level Chinese Wall . . . 3

3 Specification **4**
3.1 Methodology . . . 4
3.2 Analysis . . . 4
3.3 Requirements . . . 5

4 Design **8**
4.1 Environment Design . . . 8
4.2 System Design . . . 8

5 Product **10**
5.1 Implementation . . . 10

6 Verification & Validation **11**
6.1 Testing Suite . . . 11
6.2 Testing Constraints . . . 11

7 Results & Evaluation **12**
7.1 Evaluation Process . . . 12
7.2 Results of Evaluation . . . 12
7.3 Returning to the Research Questions . . . 12

8 Discussion & Reflection **13**
8.1 Interpreting the Results . . . 13
8.2 Reflection . . . 13
8.3 Challenges . . . 13
8.4 Limitations . . . 13
8.5 Future Work . . . 13

9 Conclusion **14**

A Appendix **16**
A.1 Ethical Approval Form . . . 16
A.2 Participant Information Sheet . . . 16
A.3 Consent Form . . . 16

A.4 Marking Scheme 17

1 Introduction

1.1 Aims

The primary aim of this project is implementation of the Chinese Wall (Brewer-Nash) Model, by leveraging the mandatory access control system introduced by Security-Enhanced Linux. SELinux is a well-known solution to security concerns threatening servers and personal devices. SELinux implementing Red Hat (IBM) have solidified themselves as the industry choice for secure hosting solutions, being a leading operating system used by over 90% of Fortune 500 companies [1] for its quality privacy guarantee.

The Chinese Wall's dynamic nature makes it an intriguing security policy, which let it stand in stark contrast from typical static security policies. For this reason or others, it has received a noticeable lack of attention in the space surrounding SELinux. While providing an implementation for SELinux, this report delves into the use cases of this specific policy, where it has succeeded in the past, and what its inclusion in SELinux allows.

"The more you know, the less you can share" is the basis for this policy, where a subject (a process, daemon or user) in the system loses more privileges as it completes its lifecycle. Conflict of Interest (`CoI`) classes dictate the main restrictions, preventing the subject from sharing sensitive information between objects (files) that are conflicting with each-other. A lattice based system is used to transition subject labels based on their read and write history accordingly, which should suffice to handle all cases of the Chinese Wall.

Previous research into both Chinese Wall effectiveness and SELinux was factored in the design process, along with hands on experimentation and prototyping of different solutions. The development of the Chinese Wall daemon should serve to open possibilities of dynamic policies in the otherwise static SELinux, and potentially lead the way in future security solutions.

To enforce this policy model in SELinux, policy modules must be compiled into the kernel via the security server and work in tandem with the designed Chinese Wall Security Daemon, developed to be capable of enforce dynamic label alterations to monitored subjects in a directory. Finally, this allows the SELinux Active Vector Cache (AVC) to handle the restrictions on a kernel level, successfully interfering in any attempted `CoI` breaches.

1.2 Objectives

1.1 Create a suitable virtual environment for SELinux.

1.2 Understand the principles of:

1.2.A The Chinese Wall security model.

1.2.B SELinux's security server & type enforcement system.

1.3 Implement a static implementation of the policy, confined by SELinux.

1.4 Develop a system daemon capable of monitoring subjects and their operations on objects.

1.5 Update said daemon to dynamically alter labels, allowing SELinux to enforce the Chinese Wall policy.

1.6 Fully integrate the Chinese Wall Daemon into SELinux's environment, including its own policy module, file hierarchy compliance and good UNIX citizenship.

1.7 Analyse the difficulties that arose with development and conclude why development in this area has been lacklustre.

1.3 Report Format

The report is structured in an sequence appropriate to the process undertook addressing this challenge, detailing background research, solution design, implementation and testing, followed by results and evaluation.

2 Background Knowledge

2.1 The Chinese Wall (Brewer-Nash) Model

2.2 Security-Enhanced Linux

2.3 Bell-LaPadula model in SELinux

2.4 Chinese Wall in UNIX

2.5 Hypervisor-Level Chinese Wall

This section should provide the grounding for your project. You will be able to refer back to this in Section 3, where you report on design decisions and methodology.

There should be a strong link to your research questions (for project types 2 and 3). There will be a strong link to the software design decisions for project type 1.

Don't just include a paragraph for each paper you read. Synthesize it and create a story line.

Review between 5 and 10 authoritative sources. For research based projects, some of these should be from peer-reviewed journals or conferences.

This section will reference literature like this [**?**]. Sloppy referencing will (1) take time and effort, and (2) lose you marks if you don't do it.

3 Specification

3.1 Methodology

A standard agile methodology was followed while completing this project. As a system with predefined capabilities was used for development and eventual execution (SELinux), various different methods were used throughout - with varying success - and prototyping was updated accordingly. Experimentation in SELinux was carried out simultaneously alongside research, allowing accumulated insight to be backed by personal experience, and a more complete and satisfying solution.

This agile process was accompanied by a simple kanban board, easily conveying the stage of the project as well as next steps and currently occupied tasks. This allowed for a more streamlined lifecycle of an idea.

A Gantt Chart was constructed in advance to demonstrate the intended project timeline:

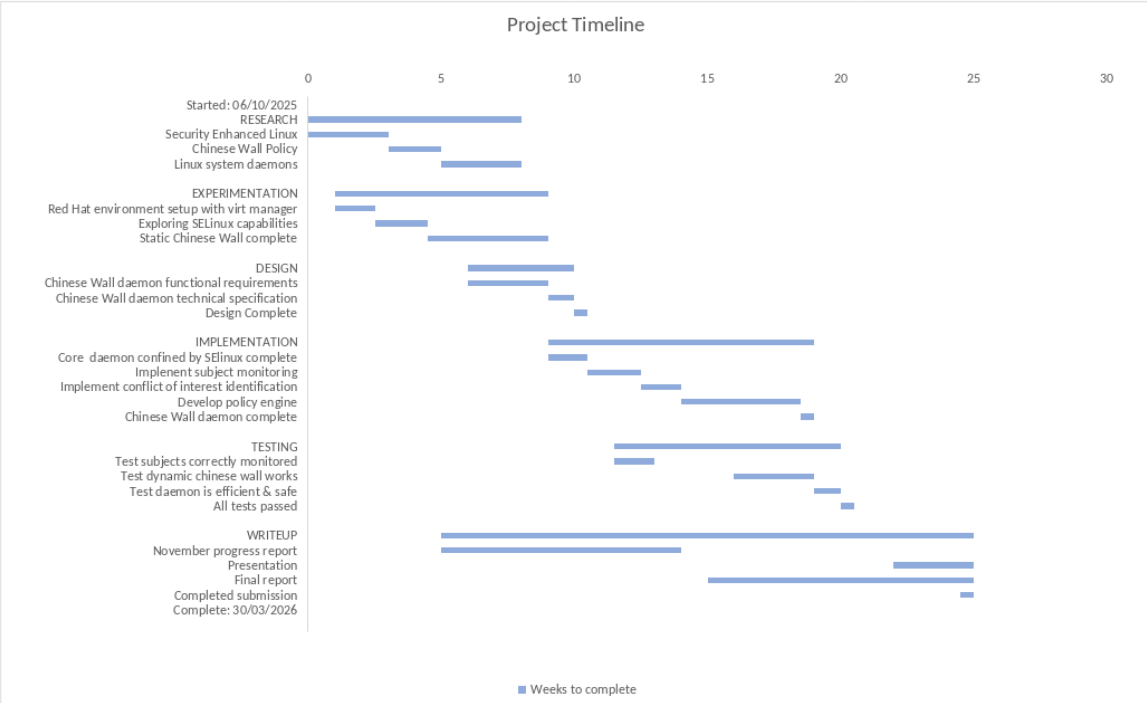

Figure 1: Project Gantt chart

3.2 Analysis

To enforce a dynamic security policy, its natural to deduce that some sort of process must be constantly running to be capable of enforcing said dynamism. During experimentation on the

boundaries of SELinux, it was surmised that the most effective way to leverage its MAC system was to change the type fragment of the labels on both subjects and objects. The user, role and category of the subjects/objects were not sufficient to enforce a security policy, as policy modules were unable to be compiled to enforce restrictions based entirely on said label fragments.

Aggressive experimentation on the type fragment of the label opened up the philosophy that would be the main premise of the Chinese Wall Daemon. Pre-compiled policy modules for all scenarios, and a daemon that altered the labels of subjects at runtime according to their behaviour and the Chinese Wall policy.

Policy modules are unable to be added to the SELinux security server at runtime, as it requires a complete re-compile of the security server and kernel. This operation is capable of taking anywhere between 4 and 8 seconds - suffice to say unrealistic at runtime for something intended as a background process.

3.3 Requirements

Requirements specified early in the agile cycle follow the MoSCoW principle. Requirements are split into: Must have, Should have, Could have and Won't have.

Functional requirements:

The Chinese Wall Daemon (`cwalld`) Must Have:

- `cwalld` **must** alter necessary configurations in the system before running.
- `cwalld` **must** monitor all operations made by subjects across a given directory.
- `cwalld` **must** make decisions based on subject label and object label aligned with the Chinese Wall policy.
- `cwalld` **must** alter the labels of subjects at upon call.
- `cwalld` **must** be accompanied by necessary policy modules to restrict labels.
- `cwalld` **should** be confined by SELinux with its own custom policy.
- `cwalld` **should** run as a service in the system with its own service declaration file.
- `cwalld` **should** constantly write operations to a log.

- `cwalld` **should** include a binary to follow and display the log in real time.
- `cwalld` **could** automatically create necessary policy modules for its runtime.
- `cwalld` **could** automatically install necessary policy modules before running
- `cwalld` **could** dynamically enforce the policy given a set of `CoI` classes as input.
- `cwalld` **won't** enforce the Chinese Wall policy on the entire system.
- `cwalld` **won't** use any sort of discretionary access control.

Non-functional requirements:

- `cwalld` **shall** run as a daemon in user-space of the computer.
- `cwalld` **shall** solely be controlled by the super-user.
- `cwalld` **shall** be written with structural, intentional and idiomatic code.
- `cwalld` **shall** be a good UNIX citizen, following common system conventions.
- `cwalld` **shall** function as intended across both x86 and ARM architecture.

Here you explain what the functional and non-functional requirements are. Explain how you prioritised them. Well-written functional requirements typically have the following characteristics:

Necessary: Although functional requirements may have different priority, every one of them needs to relate to a particular business goal or user requirement.

Concise: Use simple and easy-to-understand language without any unnecessary jargon to prevent confusion or misinterpretations.

Attainable: All requirements you include need to be realistic within the time and budget constraints set in the business requirements document.

Granular: Do not try to combine many requirements within one. The more precise and granular your requirements are, the easier it is to manage them.

Consistent: Make sure the requirements do not contradict each other and use consistent terminology.

Verifiable: It should be possible to determine whether the requirement has been met at the end of the project.

3.3.1 Functional Requirements

This is the **WHAT** of your artifact.

Functional requirements are product features that developers must implement to enable the users to achieve their goals. They define the basic system behavior under specific conditions.

Functional requirements need to be clear, simple, and unambiguous.Examples:

- The system must send a confirmation email whenever an order is placed.
- The system must allow blog visitors to sign up for the newsletter by leaving their email.
- The system must allow users to verify their accounts using their phone number.

3.3.2 Non-Functional Requirements

This is the **HOW** of your artifact. Example non-functional requirement: "When the submit button is pressed, the confirmation screen must load within 2 seconds."

— Describe all details of the design and procedures used to achieve the project objectives. Do this chronologically.

It should be detailed enough to allow for an assessment of the rigour of your process, and, in the case of research projects, in terms of how well grounded your research is in the research literature. In these cases, refer back to relevant sections in the previous chapter.

Say which software lifecycle approach you used e.g., Waterfall, Spiral, Agile.

How did you gather user requirements?

4 Design

Describe all details of the design and procedures used to achieve the project objectives. Do this chronologically.

4.1 Environment Design

Describe all policy setup etc

4.2 System Design

Show how you designed your database (if appropriate) and how you designed your system architecture, and the individual parts. Use UML and an Entity Relationship diagram

It should be detailed enough to allow for an assessment of the rigour of your process, and, in the case of research projects, in terms of how well grounded your research is in the research literature. In these cases, refer back to relevant sections in the previous chapter.

Say which software lifecycle approach you used e.g., Waterfall, Spiral, Agile.

How did you gather user requirements?

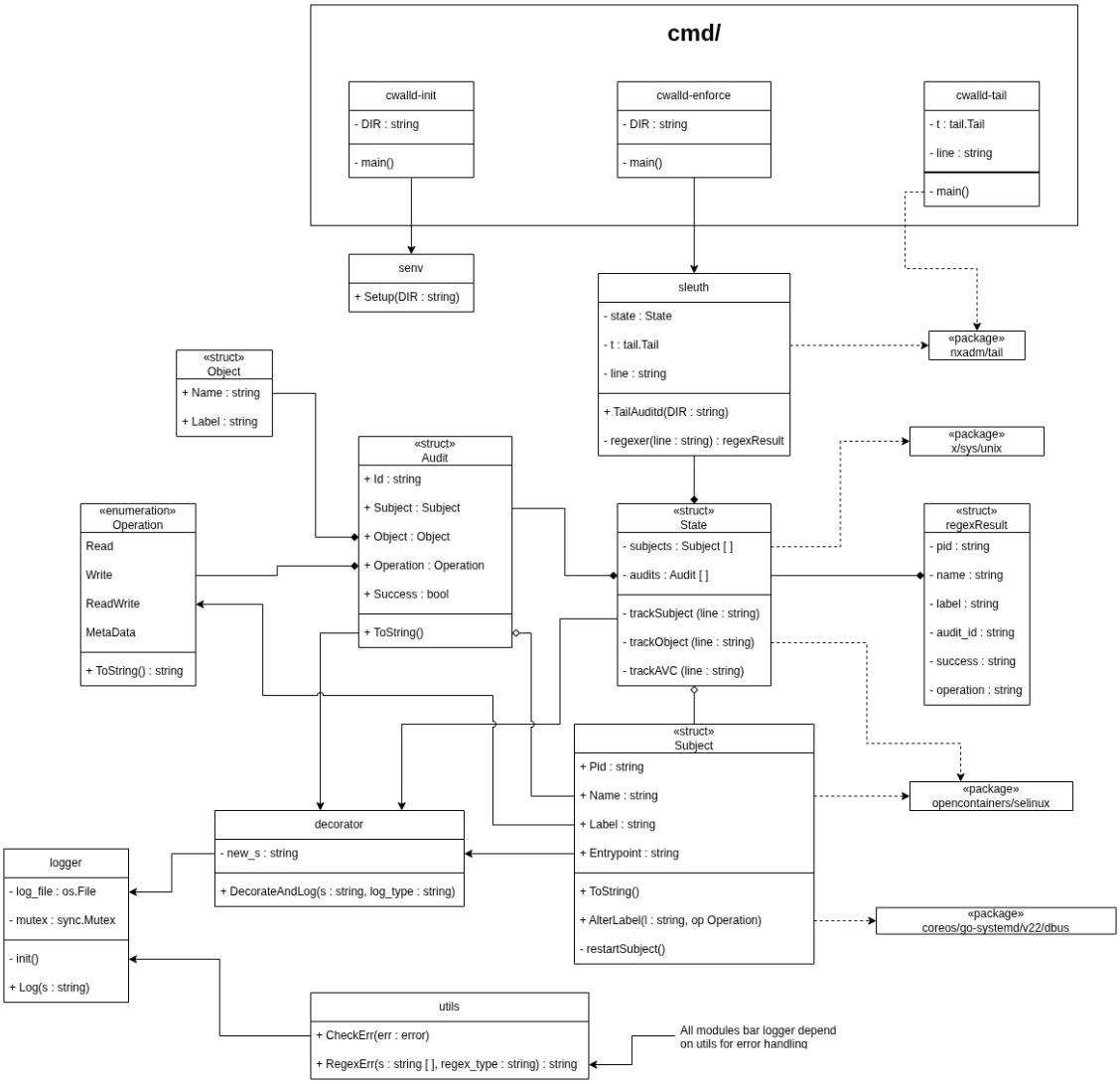

Figure 2: cwalld UML diagram

5 Product

5.1 Implementation

Provide implementation details: language used, architecture (e.g. server and client, or hub and spoke). Explain how you secured personal details.

6 Verification & Validation

6.1 Testing Suite

6.2 Testing Constraints

How did you verify that your software was debugged and worked correctly?

How did you validate that you had implemented all the functional and non-functional requirements?

7 Results & Evaluation

Results are often presented in tables, figures and other relevant illustrations. Include text that refers to these figures/tables.

7.1 Evaluation Process

If you involved humans in the evaluation, how many did you have? What can you say about the demographics of your participants? (If you did collect these)

In terms of the *user interface*, how did you carry out a usability evaluation, how did you go about doing this? How did you recruit participants?

In terms of delivering *functionality*, did you carry out user acceptance testing? (see attached guidance).

7.2 Results of Evaluation

This Section includes a direct interpretation of the gathered data and evaluation processes.

7.3 Returning to the Research Questions

Return to research questions or objectives as appropriate.

7.3.1 RQ1

It is clear from our findings that James Bond was born in Wigtown in Scotland. However, he grew up in Diss, in Norfolk. We know this because ....

7.3.2 RQ2

We were not able to answer this question from our studies, although some suggestions were made. These could not be proven.

7.3.3 Objective 1

7.3.4 Objective 2

8 Discussion & Reflection

8.1 Interpreting the Results

Here you will discuss your findings. This is especially relevant for research projects. You might interpret what the data and evaluation implies, both for future research and for practice (if appropriate).

The discussion is **not** a review of literature. You should try to compare research findings with previous work, provide explanations for your findings, discuss research findings, in terms of their contribution.

8.2 Reflection

Look back and think about what you would do differently if you were going to start the project with the knowledge you have now. Be honest about your mistakes or missteps.

8.3 Challenges

This is not the place to mention personal circumstances but rather challenges related to the work involved in the project.

8.4 Limitations

Acknowledge things like: small number of participants, software wasn't completely debugged, or whatever else went wrong and affected your project. *Include as appropriate*

8.5 Future Work

If someone else wanted to build on your project's product, what would be cool to do next?

9 Conclusion

The conclusion is similar to when a plane lands. You don't rewrite the introduction. You say something like - I addressed the problem outlined in the introduction, and I built some software to do this. I tested the software like this ADD FEW WORDS.

Summarize main findings drawn from the project work. Mention the objectives or research questions. Do not repeat points raised in the discussion and reflection Section. If applicable, you can make recommendations. The conclusion should NOT contain any references.

References

[1] Red hat enterprise linux | red hat partner connect.

A Appendix

This is where you can include your documentation.

Remember that the marker is not required to read this, but might well check to ensure that you have included product documentation, and ethical approval, as required.

A.1 Ethical Approval Form

If your project required you to do any evaluation with humans, you MUST include this. It can be downloaded from the Ethics system.

`https://local.cis.strath.ac.uk/wp/extras/ethics/index.php`

A.2 Participant Information Sheet

If your project required you to do any evaluation with humans, you MUST include this

`https://www.strath.ac.uk/ethics/informationsheetandconsentform/`

A.3 Consent Form

If your project required you to do any evaluation with humans, you MUST include this.

`https://www.strath.ac.uk/ethics/informationsheetandconsentform/`

A.4 Marking Scheme

REMEMBER TO DELETE THIS. IT IS ONLY INCLUDED FOR your INFORMATION.

11.2. Marking Schemes

There are three marking schemes which weight the assessment criteria as follows.

	Software Development Based	**Experimentation-based with Significant Software Development**	**Experiment-based**
Project progress presentation	10%	10%	10%
Product			
Implementation (including documentation as this indicates maintainability)	25	20	10
Verification and Validation	10	5	5
Product Total:	35%	25%	15%
Process			
Methodology, analysis and documentation	15	20	20
Design	10	5	5
Process Total	25%	25%	25%
Results and Evaluation			
	15%	25%	35%
Report Presentation			
	10%	10%	10%
Student Performance			
	5%	5%	5%